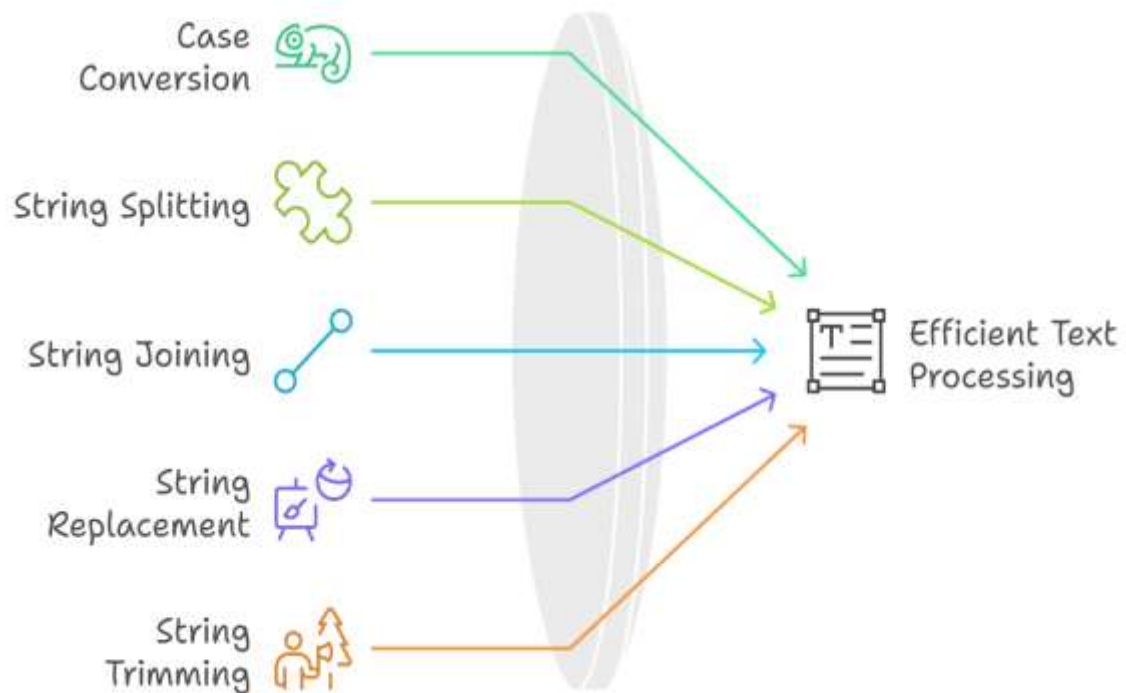


Python String Methods – A Complete Beginner-Friendly Guide

Working with strings is one of the most important skills in Python. Python provides many **built-in string methods** that make text processing easy, efficient, and powerful.

Here is a compact and clear summary of the most commonly used string methods with examples 🙌

Mastering Python String Methods



◆ 1. Changing Case

upper()

Converts all characters to uppercase.

```
"hello".upper()
```

```
# HELLO
```

lower()

Converts all characters to lowercase.

```
"PYTHON".lower()
```

```
# python
```

swapcase()

Swaps lowercase ↔ uppercase.

```
"Hello PYTHON".swapcase()
```

```
# hELLO python
```

title()

Capitalizes the first letter of every word.

```
"patience is the key".title()
```

```
# Patience Is The Key
```

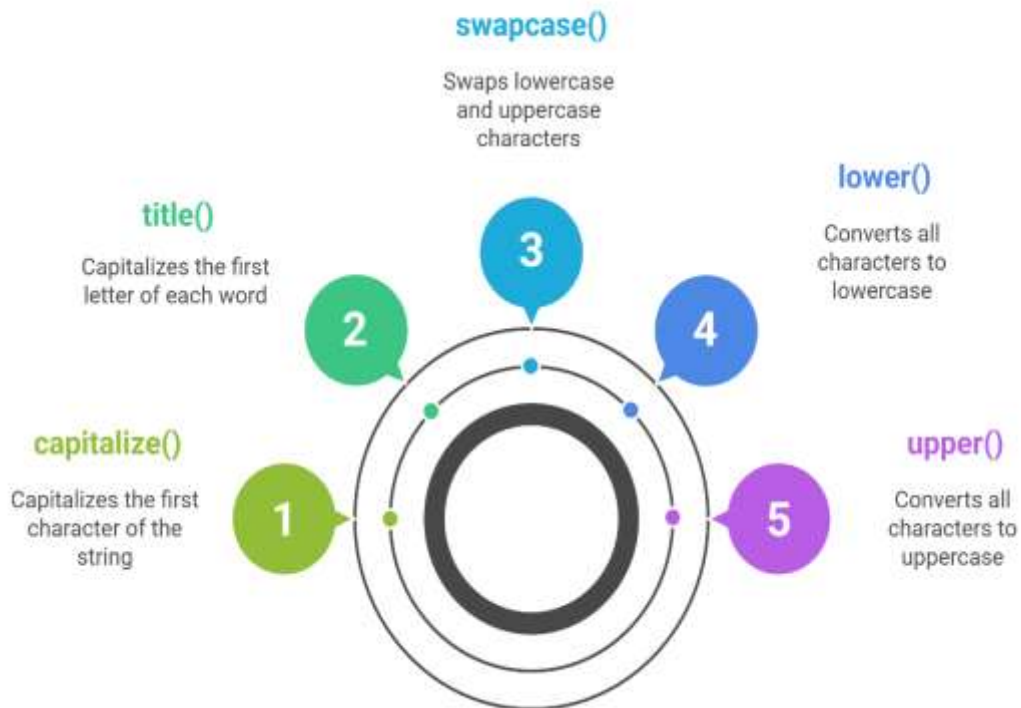
capitalize()

Capitalizes only the first character of the entire string.

```
"hello WORLD".capitalize()
```

```
# Hello world
```

String Case Conversion Methods



◆ 2. Finding Characters

`index()`

Returns the index of the first occurrence of a substring.

! Error if substring not found.

```
"Eggs are good".index("g")
```

3

`rindex()`

Same as `index()` but searches from **right** → **left**.

`find()`

Similar to `index()` but returns **-1** if substring not found.

```
"aeiou".find("x")
```

```
# -1
```

◆ 3. Counting Occurrences

count()

Counts how many times a substring appears.

```
"hello world".count("l")
```

```
# 3
```

Which string method should be used to find or count substrings?



◆ 4. Splitting Strings

split()

Splits a string into a list based on a separator (default: space).

```
"hello world".split()
```

```
# ['hello', 'world']
```

rsplit()

Splits from the **right side**.

```
"hello python world".rsplit(" ", 1)
```

```
# ['hello python', 'world']
```

◆ 5. Joining Strings

join()

Joins elements of an iterable into a single string using a separator.

```
"-".join("welcome")
```

```
# w-e-l-c-o-m-e
```

Another example:

```
"-".join(["hello", "world"])
```

```
# hello-world
```

◆ 6. Removing Unwanted Characters

strip()

Removes characters from both ends of the string.

```
"@@hello python@#".strip("@#")
```

```
# hello python
```

lstrip()

Removes characters from the **left**.

```
"###hello".lstrip("#")
```

```
# hello
```

```
rstrip()
```

Removes characters from the **right**.

```
"hello***".rstrip("*")
```

```
# hello
```

◆ 7. Replacing Characters

```
replace()
```

Replaces characters or substrings.

```
"malayalam".replace("a", "@", 2)
```

```
# m@l@yalam
```

Splitting and Joining Strings

How do I split a string into a list?

Use the `split()` method with a separator (default: space).

How do I split from the right side?

Use the `rsplit()` method with a separator and a maximum split count.

How do I join elements of an iterable into a string?

Use the `join()` method with a separator.



Conclusion

These string methods make text manipulation in Python efficient and beginner-friendly. Mastering them is essential for data cleaning, preprocessing, automation scripts, and real-world applications.

If you're learning Python, practicing these methods with different examples will boost your understanding and confidence.